

1. What is Hibernate, and why is it used in Java applications?

A: **Hibernate** is an Object-Relational Mapping (ORM) framework for Java. It is used to simplify database interaction by mapping Java objects (classes) directly to database tables, allowing developers to manipulate data using standard Java methods instead of writing repetitive SQL queries. It handles the complexity of mapping data types and managing database connections.

2. What are the key components that make up Hibernate's architecture?

A: The key components are the **Configuration** object (used for initial setup and connection properties), the **SessionFactory** (a factory for sessions), and the **Session** object (the main runtime interface for database operations). Hibernate also relies on a **Transaction** object to manage database commits and rollbacks.

3. What is the responsibility of the SessionFactory in Hibernate?

A: The **SessionFactory** is a heavy, thread-safe object responsible for reading the Hibernate configuration and holding all second-level cache data. Its primary role is to serve as a factory for creating lightweight, transaction-specific **Session** objects. Since it is expensive to create, an application usually has only one SessionFactory.

4. What is a Session in Hibernate, and how is it used for CRUD operations?

A: A **Session** is the main runtime interface between the Java application and the database. It is a lightweight object that represents a single unit of work (transaction) and acts as the **first-level cache**. For **CRUD** operations, the Session provides methods like `save()` (create/update), `get()` or `load()` (read), and `delete()` (delete).

5. How does Hibernate handle transactions internally?

A: Hibernate handles transactions by using the **Transaction** interface, which abstracts the underlying JDBC or JTA transaction. You typically begin a transaction on the Session, execute your database operations, and then explicitly call `commit()` to save changes or `rollback()` if an error occurs. This ensures **Atomicity** for all operations within the unit of work.

6. What is HQL (Hibernate Query Language), and how is it different from SQL?

A: **HQL (Hibernate Query Language)** is an object-oriented query language used by Hibernate. It is different from SQL because it operates on **entities** (Java class names and properties) rather than table names and column names. Hibernate translates the HQL query into native SQL before sending it to the database.

7. What is the Criteria API in Hibernate, and when would you use it?

A: The **Criteria API** is a programmatic way to build dynamic queries in Hibernate using Java objects and methods instead of string-based HQL or SQL. You would use it when you need to build complex queries whose conditions (WHERE clauses) are constructed at **runtime** based on user input or application logic.

8. What is the purpose of the @Entity annotation in Hibernate?

A: The **@Entity** annotation is placed on a Java class to mark it as a persistent class that corresponds to a table in the database. It signals to Hibernate that this class's state should be tracked and managed (mapped) to database records.

9. What is cascading in Hibernate, and why is it important in entity relationships?

A: **Cascading** is a feature that defines how specific lifecycle events (like save, update, or delete) on a parent entity should automatically affect its associated child entities. It is important because it allows developers to maintain data integrity and avoid manual code to synchronize the state of related objects.

10. What is a composite key in Hibernate, and how can it be implemented?

A: A **composite key** is a primary key that consists of two or more columns combined. It can be implemented in Hibernate using a separate **ID class** (annotated with @Embeddable and used with @EmbeddedId) or by using the **@IdClass** annotation on the main entity. The ID class must implement Serializable.

11. Explain Hibernate's object states: Transient, Persistent, and Detached.

A:

- **Transient:** A new Java object, not yet associated with a Session, no representation in the database.
- **Persistent:** The object is associated with a Session, and its changes are tracked by Hibernate.
- **Detached:** The object was Persistent, but the Session has been closed; its changes are no longer automatically tracked by Hibernate.

12. What is lazy loading in Hibernate, and how does it work by default?

A: **Lazy loading** is a performance optimization where the related data (like a list of orders associated with a customer) is not loaded from the database until it is explicitly accessed by the application code. It works by default for collection mappings and is achieved by injecting a **proxy object** that executes the necessary SQL query only when the data is requested.

13. What is the role of the Configuration class in setting up Hibernate?

A: The **Configuration** class is the starting point for setting up Hibernate. Its primary role is to load the necessary configuration parameters (database URL, dialect, connection pool settings) and map the persistent entity classes. It is used to bootstrap the entire Hibernate environment before the SessionFactory is created.

14. How does Hibernate protect against SQL injection (basic explanation of parameter binding)?

A: Hibernate protects against **SQL injection** by using **parameter binding**. Instead of placing user-provided data directly into the SQL string, Hibernate uses placeholders (like ? or named parameters). The user input is always sent to the database separately as a data value, ensuring it is never treated as executable SQL code.

15. What are the differences between the `get()` and `load()` methods in Hibernate?

A: `get()` immediately hits the database, returning the actual object or null if the record is not found. `load()` returns a **proxy object** immediately without hitting the database; it throws an exception if the data is accessed later and the record is not found. `get()` is safer for checking existence; `load()` is faster if you know the object exists.

JAVA Interview Buddy